
最高分子序列

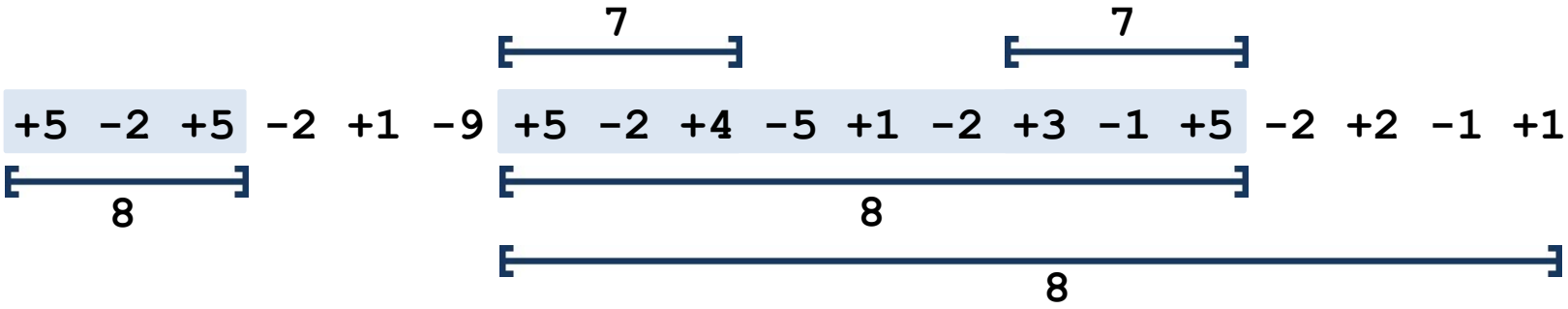

最高分-子序列

输入值：一个序列 $(a_1, \ldots, a_n) \in R$

+5 -2 +5 -2 +1 -9 +5 -2 +4 -5 +1 -2 +3 -1 +5 -2 +2 -1 +1

输入值：一个序列 $(a_1, \ldots, a_n) \in R$

输出值：一个子序列 $(a_i, \ldots, a_j)$，使得函数 $f(i,j)$ 最大，$f(i,j) = \sum_{h=i}^{j} a_h$

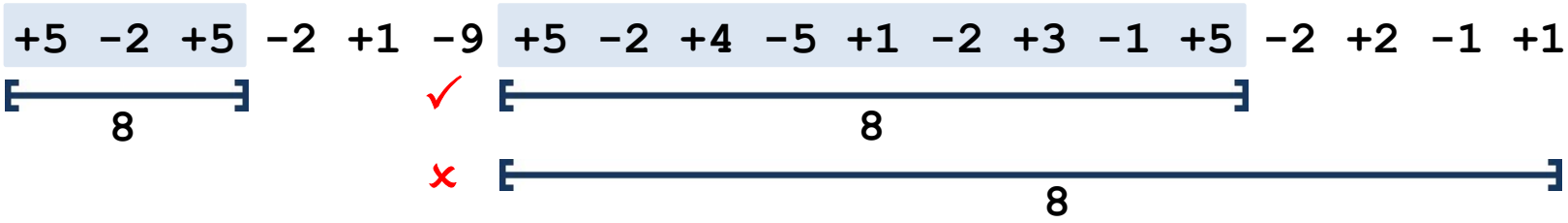


输入序列的子序列最大分值是：8



输入值：一个序列 $(a_1, \ldots, a_n) \in R$

输出值：一个子序列 $(a_i, \ldots, a_j)$，使得函数 $f(i,j)$ 最大，$f(i,j) = \sum_{h=i}^{j} a_h$

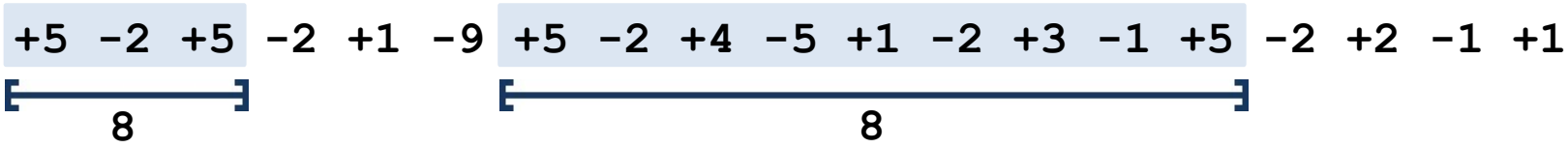


输入序列的子序列最大分值是：8

最短原则：在几个子序列同时拥有最高分时，如果某一个完全包含在另一之内，则只返回被包含的那一个。

输入值：一个序列 $(a_1, \ldots, a_n) \in R$

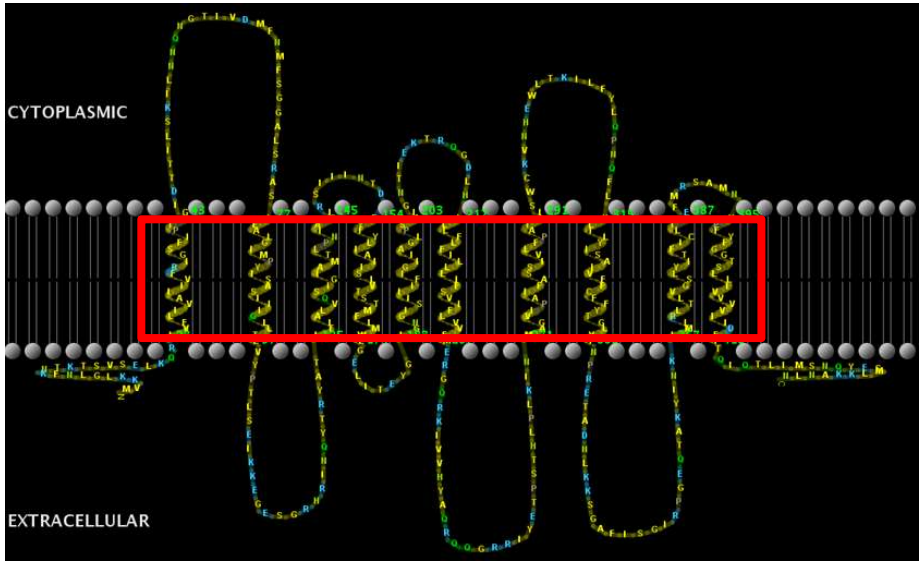
输出值：一个子序列 $(a_i, \ldots, a_j)$，使得函数 $f(i,j)$ 最大，$f(i,j) = \sum_{h=i}^{j} a_h$



输入序列的子序列最大分值是：8

最短原则：在几个子序列同时拥有最高分时，如果某
一个完全包含在另一之内，则只返回被包含的那一个。

生物学应用：（1）预测蛋白质序列跨膜区域。根据氨基酸的亲疏水性不同，将字符串序列转化为实数序列，疏水氨基酸[0,5]，亲水氨基酸[-5,0]。



		最高分子序列									
S =	S	L	V	F	I	V	E	T	A	D	N
S′=	-0.8	3.8	4.2	2.8	4.5	4.2	-3.5	-0.7	1.8	-3.5	-3.5
		疏水片段									

生物学应用：（2）预测DNA序列中富含GC的区域。比如寻找CpG岛。CpG岛主要位于基因的启动子和第一外显子区域，约有60%以上基因的启动子含有CpG岛。因此，在大规模测序中每发现一个CpG岛，则预示着在此可能有基因的存在。





转化：G, C给正分；A,T给负分。

		最	高	分	子	序	列				
S =	T	G	G	C	C	G	A	T	A	G	T
S′=	-1	1	1	1	1	1	-1	-1	-1	1	-1
		G	C	富	集	区					

输入值：一个序列 $(a_1, \dots, a_n) \in R$

输出值：一个子序列 $(a_i, \dots, a_j)$，使得函数 $f(i,j)$ 最大，$f(i,j) = \sum_{h=i}^{j} a_h$

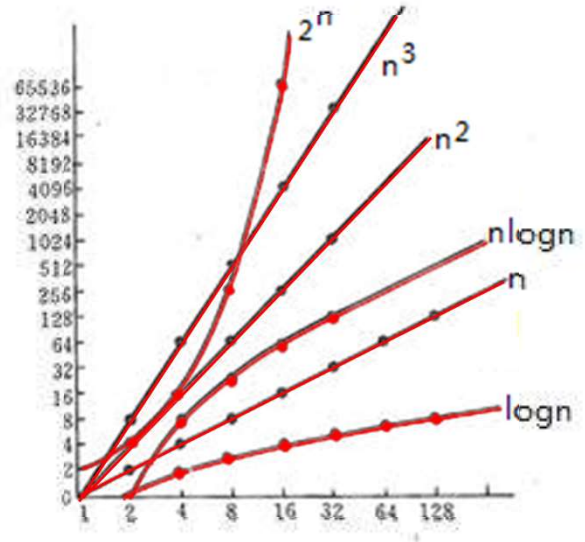
Naïve算法：对于所有 $i \le j \in [1,n]$，计算 $f(i,j)$，再找出最大值对应的 (i,j)。

所有可能的 (i,j) 组合的数量，即计算 $f(i,j)$ 的次数=

$$n + (n-1) + (n-2) + \cdots + 1 = \frac{n \times (n+1)}{2} = O(n^2)$$

计算一次 $f(i,j)$ 所需的步骤：$O(n)$

→Naïve算法的总运算步骤为：$O(n^3)$



输入值：一个序列 $(a_1, \dots, a_n) \in R$

输出值：一个子序列 $(a_i, \dots, a_j)$，使得函数 $f(i,j)$ 最大，$f(i,j) = \sum_{h=i}^{j} a_h$

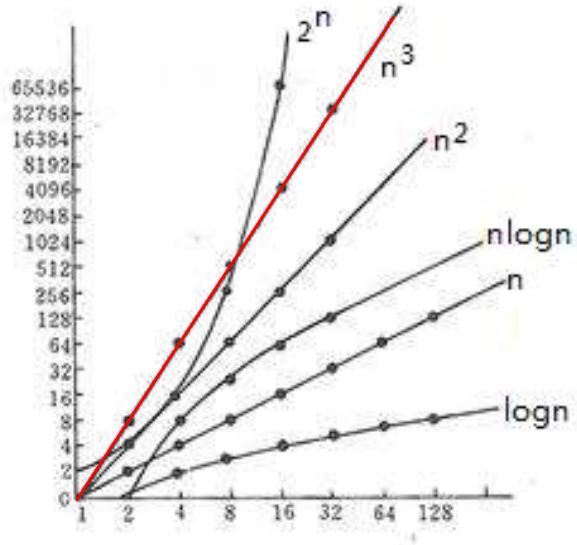
Naïve算法：对于所有 $i \le j \in [1,n]$，计算 $f(i,j)$，再找出最大值对应的 (i,j)。

所有可能的 (i,j) 组合的数量，即计算 $f(i,j)$ 的次数=

$$n + (n-1) + (n-2) + \cdots + 1 = \frac{n \times (n+1)}{2} = O(n^2)$$

计算一次 $f(i,j)$ 所需的步骤：$O(n)$

→Naïve算法的总运算步骤为：$O(n^3)$



输入值：一个序列 $(a_1, \dots, a_n) \in R$

输出值：一个子序列 $(a_i, \dots, a_j)$，使得函数 $f(i,j)$ 最大，$f(i,j) = \sum_{h=i}^{j} a_h$

Naïve算法：对于所有 $i \le j \in [1,n]$，计算 $f(i,j)$，再找出最大值对应的 (i,j)。

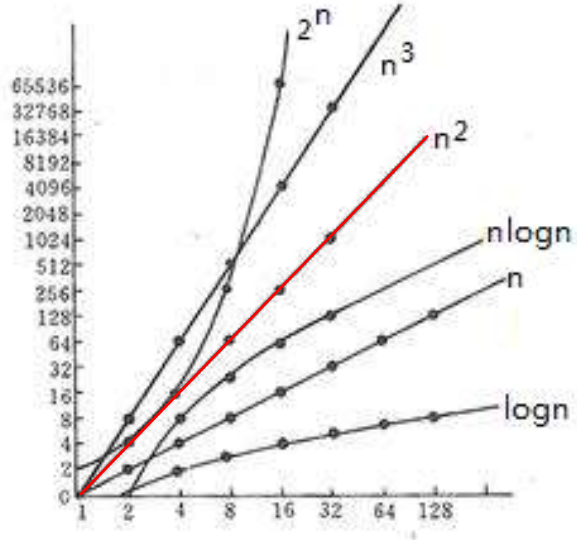
所有可能的 (i,j) 组合的数量，即计算 $f(i,j)$ 的次数=

$$n + (n-1) + (n-2) + \cdots + 1 = \frac{n \times (n+1)}{2} = O(n^2)$$

计算一次 $f(i,j)$ 所需的步骤：$O(n)$

→Naïve算法的总运算步骤为：$O(n^3)$

→动态算法的总运算步骤为：$O(n^2)$



输入值：一个序列 $(a_1, \dots, a_n) \in R$

输出值：一个子序列 $(a_i, \dots, a_j)$，使得函数 $f(i,j)$ 最大，$f(i,j) = \sum_{h=i}^{j} a_h$

Naïve算法：对于所有 $i \le j \in [1,n]$，计算 $f(i,j)$，再找出最大值对应的 (i,j)。

所有可能的 (i,j) 组合的数量，即计算 $f(i,j)$ 的次数=

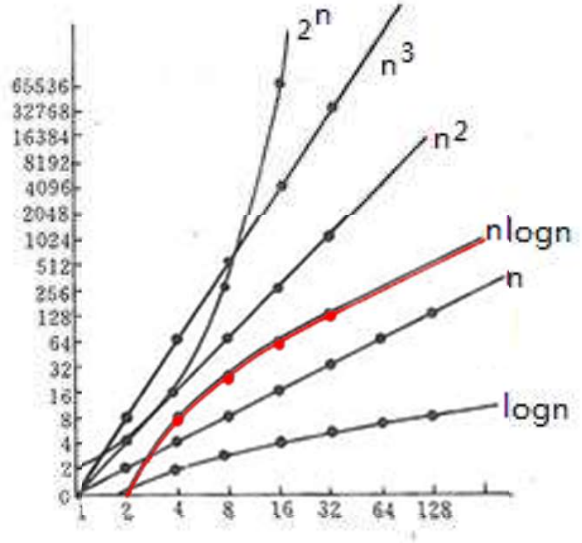
$$n + (n-1) + (n-2) + \cdots + 1 = \frac{n \times (n+1)}{2} = O(n^2)$$

计算一次 $f(i,j)$ 所需的步骤：$O(n)$

→Naïve算法的总运算步骤为：$O(n^3)$

→动态算法的总运算步骤为：$O(n^2)$

→分而治之法的总运算步骤为：$O(n \log n)$



输入值：一个序列 $(a_1, \ldots, a_n) \in R$

输出值：一个子序列 $(a_i, \ldots, a_j)$，使得函数 $f(i,j)$ 最大，$f(i,j) = \sum_{h=i}^{j} a_h$

Naïve算法：对于所有 $i \leq j \in [1,n]$，计算 $f(i,j)$，再找出最大值对应的 (i,j)。

所有可能的 (i,j) 组合的数量，即计算 $f(i,j)$ 的次数=

$$n + (n-1) + (n-2) + \cdots + 1 = \frac{n \times (n+1)}{2} = O(n^2)$$

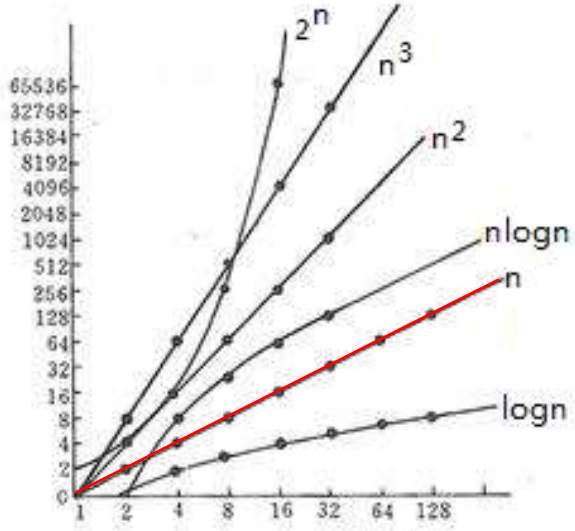
计算一次 $f(i,j)$ 所需的步骤：$O(n)$

→Naïve算法的总运算步骤为：$O(n^3)$

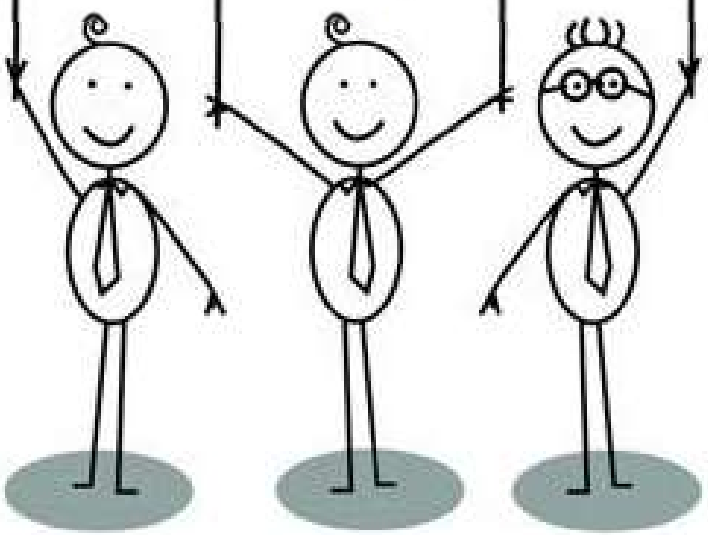
→动态算法的总运算步骤为：$O(n^2)$

→分而治之法的总运算步骤为：$O(n \log n)$

→聪明算法的总运算步骤为：$O(n)$



序列算法



理论基石

